Polytech Dijon

Rapport de Travaux pratiques

Carte sans contact MIFARE Classic

Fabio CUELLAR
Anice GUÉNIAT

Année universitaire 2026-2027

1. Introduction

Ce TP s'inscrit dans le projet de carte multiservice pour les étudiants de Polytech Dijon. La carte utilisée est une NXP MIFARE Classic 1K. Elle doit contenir l'identité de l'étudiant (nom et prénom) ainsi qu'un porte-monnaie électronique, c'est-à-dire un nombre d'unités utilisable à la cafétéria ou pour les photocopies.

Nous avons développé sous Qt Creator (kit Desktop Qt 6.11.2 MinGW 64 bits) une interface graphique qui permet de lire et de modifier ces informations à l'aide d'un lecteur USB ODALID. Dans ce rapport, nous présentons d'abord le fonctionnement de la librairie ODALID, puis nous expliquons chaque fonction du fichier tp1_com.cpp.

2. Utilisation de la librairie ODALID

2.1 Principe de communication

Notre programme ne communique pas directement avec la carte. Il appelle les fonctions de la librairie ODALID, qui envoient des commandes au lecteur par une liaison série. Le lecteur USB est de type CDC, il apparaît donc sur le PC comme un port COM virtuel. C'est le lecteur qui produit le champ radiofréquence à 13,56 MHz. Ce champ alimente la carte, qui n'a pas de pile, et sert aux échanges selon la norme ISO/IEC 14443 type A (106 kbit/s). Le lecteur contient aussi un Secure Element dans lequel sont stockées les clés de la carte.

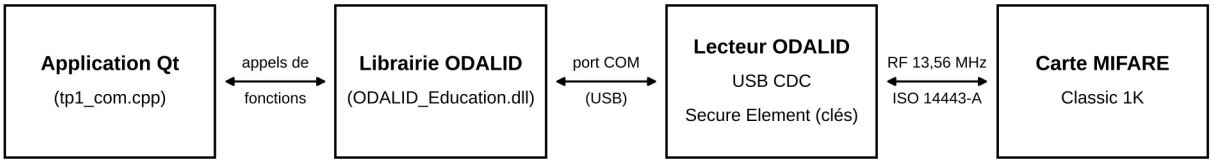


Figure 1 : chaîne de communication entre l'application et la carte

2.2 Intégration dans le projet Qt

La librairie est fournie sous forme d'une DLL (ODALID_Education.dll), d'une bibliothèque d'import pour MinGW (libODALID_Education.a) et de fichiers d'en-tête. Ces fichiers sont placés dans un dossier LIB, au même niveau que le projet, et déclarés dans le fichier TP1Com_sans_fils.pro :

```
unix|win32: LIBS += -L$$PWD/../LIB/ -lODALID_Education
INCLUDEPATH += $$PWD/../LIB
DEPENDPATH += $$PWD/../LIB
```

Les fonctions sont réparties dans plusieurs fichiers d'en-tête, que nous incluons au début de tp1_com.cpp : Sw_Device.h pour piloter le lecteur, Sw_ISO14443A-3.h pour la détection de la carte, Sw_Mf_Classic.h pour les commandes MIFARE Classic, MfErrNo.h pour les codes d'erreur, Hardware.h et TypeDefs.h pour les types et la structure ReaderName.

2.3 Le lecteur et les codes de retour

Toutes les fonctions prennent en premier paramètre un pointeur vers une structure ReaderName qui décrit le lecteur. Avant d'ouvrir la communication, nous indiquons le type de lecteur (ReaderCDC pour un lecteur USB, ReaderTCP pour un lecteur Ethernet) et le port à utiliser (champ device). La librairie complète ensuite cette structure, par exemple avec la version du firmware.

Les fonctions renvoient un code de type int16_t. La valeur MI_OK (0) signifie que l'opération a réussi. Une valeur négative correspond à une erreur, par exemple MI_NOTAGERR (-1) quand aucune carte ne répond, MI_AUTHERR (-4) en cas d'erreur d'authentification ou MI_SER_TIMEOUT_ERR (-246) quand le lecteur ne répond pas. La fonction GetErrorMessage() donne le message correspondant à un code.

Les diagrammes de séquences du sujet donnent l'ordre des appels : ouverture de la communication (OpenCOM), lecture de la version (Version), activation du champ RF (RF_Power_Control), détection de la carte (ISO14443_3_A_PollCard), lectures et écritures (fonctions Mf_Classic_...), puis coupure du champ et fermeture de la communication (RF_Power_Control et CloseCOM).

2.4 Mémoire de la carte et authentification

La MIFARE Classic 1K contient 1 Ko de mémoire, répartie en 16 secteurs de 4 blocs de 16 octets. Le dernier bloc de chaque secteur (sector trailer) contient la clé A, les conditions d'accès et la clé B. Les conditions d'accès indiquent quelle clé autorise chaque opération. Avant de lire ou d'écrire un bloc, il faut donc s'authentifier sur son secteur avec la bonne clé.

Un bloc peut servir de bloc de données (16 octets libres, utilisés ici pour le nom et le prénom) ou de bloc valeur. Un bloc valeur contient un entier de 4 octets enregistré plusieurs fois, ce qui permet de détecter les erreurs. La carte peut l'incrémenter, le décrémenter ou le restaurer, puis transférer le résultat dans un bloc. Nous l'utilisons pour le compteur d'unités.

Dans ce TP, les clés sont déjà chargées dans le Secure Element du lecteur, à un index compris entre 0 et 15. Les fonctions MIFARE de la librairie prennent donc trois paramètres pour l'authentification : Auth (TRUE pour que le lecteur s'authentifie automatiquement), AuthKey (AuthKeyA ou AuthKeyB) et KeyIndex (index de la clé dans le lecteur). L'organisation de la carte est imposée par le sujet :

Secteur	Bloc	Contenu	Clés
2	8	« Identité » (nom de l'application)	Lecture : clé A, index 2. Écriture : clé B, index 2.
	9	Prénom	
	10	Nom	
	11	Clé A, conditions d'accès, clé B	
3	12	« Porte Monnaie » (nom de l'application)	Lecture et décrémentation : clé A, index 3. Écriture et incrémentation : clé B, index 3.
	13	Sauvegarde du compteur	
	14	Compteur d'unités	
	15	Clé A, conditions d'accès, clé B	

3. Explication du fichier tp1_com.cpp

La fenêtre est gérée par la classe TP1_Com, qui hérite de QMainWindow. Chaque bouton de l'interface (fichier tp1_com.ui) est associé à un slot nommé on_<nom du bouton>_clicked(). Qt relie automatiquement ces slots aux boutons lors de l'appel à setupUi(), grâce à QMetaObject::connectSlotsByName().

Au début du fichier, nous déclarons des variables globales : MonLecteur, de type ReaderName, utilisée par tous les slots, et les tableaux atq, sak et uid (avec leur longueur uid_len), qui reçoivent les informations de la carte lors de sa détection. Le tableau suivant récapitule les fonctions de la librairie appelées par chaque slot :

Slot	Fonctions ODALID appelées	Blocs	Clés
on_Connect_clicked()	OpenCOM, Version, RF_Power_Control		
on_Disconnect_clicked()	RF_Power_Control, LEDBuzzer, CloseCOM		
on_Selection_clicked()	ISO14443_3_A_PollCard, Mf_Classic_Read_Block, Mf_Classic_Read_Value	10, 9, 14	A
on_Mise_a_jour_clicked()	Mf_Classic_Write_Block, Mf_Classic_Read_Block	10, 9	B puis A
on_Charger_clicked()	Mf_Classic_Increment_Value, Mf_Classic_Read_Value	14	B puis A
on_Payer_clicked()	Mf_Classic_Decrement_Value, Mf_Classic_Read_Value	14	B puis A

3.1 Constructeur et destructeur

Le constructeur appelle ui->setupUi(this), qui crée les éléments de l'interface et connecte les slots. Le destructeur libère l'objet ui. Ces deux fonctions n'utilisent pas la librairie.

3.2 on_Connect_clicked()

Cette fonction ouvre la communication avec le lecteur. Nous remplissons d'abord MonLecteur (Type = ReaderCDC et device = 0), puis nous appelons OpenCOM(&MonLecteur). Le code de retour est affiché dans la console avec qDebug() : il vaut 0 (MI_OK) si l'ouverture a réussi. La fonction Version() récupère ensuite la version du lecteur dans MonLecteur.version, et RF_Power_Control(&MonLecteur, true, 1) active le champ RF qui alimente la carte. Pour finir, la version est affichée dans la zone Affiche.

3.3 on_Disconnect_clicked()

Cette fonction ferme proprement la communication avec le lecteur. Elle coupe d'abord le champ RF avec RF_Power_Control(&MonLecteur, FALSE, 0), éteint les LED et le buzzer avec LEDBuzzer(&MonLecteur, LED_OFF), puis ferme le port avec CloseCOM(&MonLecteur). L'appel à qApp->quit() ferme ensuite l'application, comme le bouton Quitter du tutoriel.

3.4 on_Selection_clicked()

Cette fonction détecte la carte et affiche son contenu. ISO14443_3_A_PollCard() envoie une requête à la carte, gère l'anticollision puis sélectionne la carte. L'ATQA, le SAK et l'UID sont alors stockés dans atq, sak et uid. Nous lisons ensuite le nom (bloc 10) et le prénom (bloc 9) avec Mf_Classic_Read_Block() et la clé A d'index 2, puis le compteur (bloc 14) avec Mf_Classic_Read_Value() et la clé A d'index 3. Chaque valeur n'est affichée que si la fonction a renvoyé MI_OK. Le nom et le prénom sont convertis en texte avec un cast (char*), et le compteur avec QString::number().

3.5 on_Mise_a_jour_clicked()

Cette fonction écrit dans la carte le nom et le prénom saisis dans l'interface. Le texte est récupéré avec toPlainText(), converti en UTF-8 avec toUtf8(), puis copié dans un tableau de 16 octets avec strncpy, comme indiqué dans le tutoriel Qt. Si le texte fait moins de 16 octets, strncpy complète le tableau avec des zéros. Le nom est écrit dans le bloc 10 et le prénom dans le bloc 9 avec Mf_Classic_Write_Block() et la clé B d'index 2. Nous relisons ensuite ces deux blocs avec la clé A pour afficher ce qui a vraiment été enregistré.

Cette fonction, comme les deux suivantes, n'appelle pas ISO14443_3_A_PollCard() : il faut donc d'abord sélectionner la carte avec le bouton « Selection de la carte ».

3.6 on_Charger_clicked()

Cette fonction ajoute des unités au porte-monnaie. Le nombre d'unités à ajouter est lu dans la QSpinBox Affiche_Charger. La fonction Mf_Classic_Increment_Value(&MonLecteur, TRUE, 14, val, 14, AuthKeyB, 3) ajoute ensuite cette valeur au bloc 14 avec la clé B d'index 3. Le cinquième paramètre indique le bloc dans lequel le résultat est transféré, ici le bloc 14 lui-même. Le nouveau solde est ensuite relu avec Mf_Classic_Read_Value() et affiché.

3.7 on_Payer_clicked()

Cette fonction est construite comme la précédente, mais elle retire des unités au lieu d'en ajouter. Le nombre d'unités est lu dans Affiche_Payer, puis retiré du bloc 14 avec Mf_Classic_Decrement_Value(). Nous utilisons ici la clé B, alors que le sujet prévoit la clé A pour la décrémentation. Cela fonctionne, car les conditions d'accès d'un bloc valeur autorisent la décrémentation avec les deux clés, mais il serait plus logique d'utiliser la clé A comme le demande le sujet.

4. Conclusion

Ce TP nous a permis de comprendre comment une application communique avec une carte sans contact par l'intermédiaire d'un lecteur et de sa librairie. La librairie ODALID simplifie beaucoup le travail : il suffit de décrire le lecteur, d'appeler les fonctions dans le bon ordre et de vérifier les codes de retour, sans gérer directement les échanges radio ni les clés. Notre application permet de lire et de modifier l'identité de l'étudiant, et de créditer ou débiter son porte-monnaie.

Il reste cependant des améliorations à apporter. La sauvegarde du compteur dans le bloc 13 avec Mf_Classic_Restore_Value() n'est pas encore faite, le lecteur ne signale pas les transactions avec sa LED ou son buzzer comme le demande le sujet, et les codes de retour ne sont pas tous vérifiés.